Taller: Docker Compose

Materia:

Sistemas distribuidos

Estudiante:

Brayan Esmid Cruz Chate

Docente:

LEIDY MARIANI DORADO FLÓREZ

Corporación Universitaria Comfacauca - Unicomfacauca

Popayán

2026

Este documento presenta una guía académica detallada sobre el diseño, análisis y exploración de un sistema distribuido mediante el uso de **Docker Compose**. El sistema propuesto es una aplicación de gestión de tareas (To-Do App) que ejemplifica los principios fundamentales de la arquitectura de servicios.

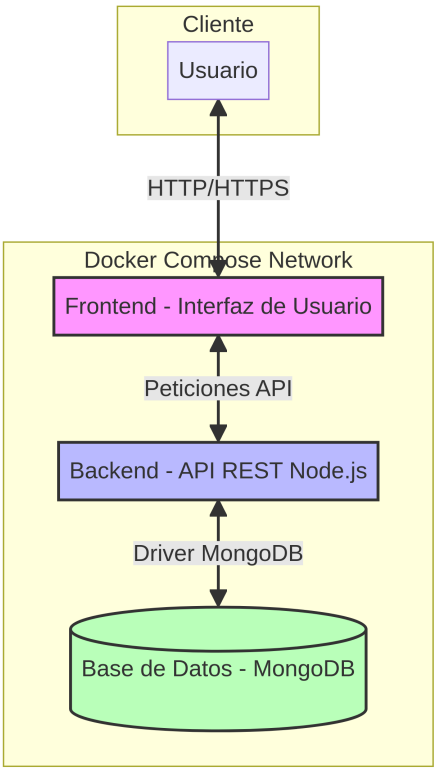
1. Diseño del Sistema

El sistema consiste en una aplicación de gestión de tareas diseñada bajo una arquitectura distribuida. Este enfoque permite que cada componente del sistema funcione de manera independiente, facilitando su mantenimiento y escalabilidad. El sistema se compone de tres servicios principales orquestados mediante un entorno de contenedores.

Servicio	Tecnología	Función Principal
Frontend	JS/ CSS / HTML	Interfaz de usuario para la interacción directa.
Backend	Node.js (Express)	Lógica de negocio y gestión de la API REST.
Base de Datos	MongoDB	Almacenamiento persistente de las tareas.

Diagrama de Arquitectura

A continuación, se presenta la estructura de comunicación entre los servicios dentro de la red interna de Docker:



Comunicación entre Servicios

En un entorno de Docker Compose, la comunicación se realiza a través de una red virtual privada. Cada servicio puede identificar a los demás utilizando su nombre de servicio como si fuera un dominio (hostname). Por ejemplo, el backend se conecta a la base de datos mediante la dirección mongodb://database:27017.

Escenarios de Fallo

La robustez de un sistema distribuido se mide por su capacidad de respuesta ante fallos en sus componentes individuales.

- **Fallo del Frontend:** El sistema sigue operativo internamente, pero el usuario pierde el acceso visual. Los datos permanecen seguros en la base de datos.
- **Fallo del Backend:** La interfaz de usuario carga, pero las acciones de guardar o eliminar tareas fallarán. El frontend está programado para mostrar un mensaje de error adecuado al usuario.
- **Fallo de la Base de Datos:** El backend no podrá completar las solicitudes de persistencia. Es el escenario más crítico, ya que la aplicación pierde su funcionalidad principal de almacenamiento.

2. Análisis del Sistema

Roles de cada Servicio

Cada componente del sistema tiene una responsabilidad única y bien definida:

1. **Frontend:** Se encarga de la presentación y la experiencia del usuario, capturando las entradas y mostrando los resultados.
2. **Backend:** Actúa como intermediario, validando los datos, aplicando reglas de negocio y gestionando la seguridad antes de interactuar con la base de datos.
3. **Base de Datos:** Garantiza que la información persista incluso si los contenedores del frontend o backend se reinician.

Ventajas de la División en Múltiples Servicios

Dividir un sistema monolítico en servicios independientes ofrece beneficios significativos:

- **Escalabilidad:** Es posible aumentar los recursos solo para el servicio que lo necesite (por ejemplo, más instancias del backend durante picos de uso).
- **Mantenibilidad:** Los errores se localizan más fácilmente y las actualizaciones de un servicio no requieren detener todo el sistema.
- **Independencia Tecnológica:** Cada servicio puede ser desarrollado con la tecnología que mejor se adapte a su función específica.

Comunicación de Contenedores en Docker Compose

Docker Compose utiliza un sistema de DNS interno. Cuando se inicia el proyecto, se crea una red automática donde cada contenedor recibe una dirección IP interna. Los servicios se encuentran entre sí resolviendo el nombre asignado en el archivo de configuración, lo que elimina la necesidad de gestionar direcciones IP manuales.

3. Exploración Técnica

Variables de Entorno en Docker

Las variables de entorno son parámetros de configuración externos al código fuente. Permiten que la misma imagen de contenedor funcione en diferentes entornos (desarrollo, pruebas, producción) simplemente cambiando sus valores. En Docker Compose, se definen bajo la sección environment.

Conexión Backend y Base de Datos

La conexión se establece mediante un **Driver de Base de Datos** (como Mongoose en Node.js). El backend utiliza una cadena de conexión (URI) que incluye el protocolo, el nombre del servicio de la base de datos y las credenciales necesarias.

Posibles Errores de Conexión

Durante el desarrollo de sistemas distribuidos, es común enfrentarse a los siguientes problemas:

- **ECONNREFUSED:** El servicio de la base de datos aún no ha terminado de iniciar cuando el backend intenta conectarse.
- **ENOTFOUND:** Existe un error tipográfico en el nombre del servicio dentro de la cadena de conexión.
- **Timeouts:** La red está saturada o hay reglas de firewall (puertos cerrados) que impiden el flujo de datos entre contenedores.